Code Explanation

Unit Test for Policy Data Transformation

The provided code is a unit test for the `transform_policy_data` function in the `src.transformation` module. This test ensures that the function correctly transforms policy data by verifying the joins, aliases, and filters applied to the input data.

Overview
The test uses Pytest and the `unittest.mock` library to mock the source tables and the `transform_policy_data` function dependencies. It tests the function by verifying that the correct joins, aliases, and filters are applied to the input data.

Step 1: Creating Mock Source Tables
This step creates mock source tables for testing the `transform_policy_data` function. The `mock_source_tables` fixture creates a dictionary of mock DataFrames, each representing a source table.

```
@pytest.fixture
def mock_source_tables():
    tables = {}
    for table_name in ["T_TX_REQUEST_POLICY", "T_TX_BASIC", "T_TX_RE
LATION",
                       "T_TX_REQUEST", "T_TX_DTL_BENEFICIARY_CHANGE"
]:
        df = MagicMock(spec=DataFrame)
        df.alias.return_value = df
        df.join.return_value = df
        df.select.return_value = df
        df.filter.return_value = df
        tables[table_name] = df
    return tables
```

Step 2: Configuring Mocks and Calling the Function
This step configures the mocks for the `transform_policy_data` function dependencies and calls the function with the mock source tables.

```
with patch('src.transformation.DEFAULT_VALUES', {
    "DATA_TYPE": "TEST_TYPE",
    "POLICY_TYPE": "TEST_POLICY",
    "MC_CRNCY": "USD"
}), patch('src.transformation.col') as mock_col,
     patch('src.transformation.lit') as mock_lit,
     patch('src.transformation.current_timestamp') as mock_timestamp
,
     patch('src.transformation.monotonically_increasing_id') as mock
_id:

    # Configure mocks
    mock_col.return_value = MagicMock()
    mock_lit.return_value = MagicMock()
    mock_timestamp.return_value = MagicMock()
    mock_id.return_value = MagicMock()
```

```
    # Call the function
    result = transform_policy_data(mock_source_tables)
```

Step 3: Verifying Aliases and Joins
This step verifies that the correct aliases are applied to the source tables and that the joins are made.

```
trp = mock_source_tables["T_TX_REQUEST_POLICY"]
ttb = mock_source_tables["T_TX_BASIC"]
tr = mock_source_tables["T_TX_REQUEST"]
ttr = mock_source_tables["T_TX_RELATION"]
ttdbc = mock_source_tables["T_TX_DTL_BENEFICIARY_CHANGE"]

trp.alias.assert_called_once_with("trp")
ttb.alias.assert_called_once_with("ttb")
tr.alias.assert_called_once_with("tr")
ttr.alias.assert_called_once_with("ttr")
ttdbc.alias.assert_called_once_with("ttdbc")

# Verify joins and transformations
assert trp.join.called
```

Step 4: Verifying the Final Result
This step verifies that the final result is the filtered DataFrame.

```
assert result.filter.called

# The final result should be the filtered DataFrame
assert result == mock_source_tables["T_TX_REQUEST_POLICY"].alias().j
oin().join().join().join().select().filter()
```